Hybrid Framework for Deepfake Detection: A Diagnostic Study of Spatial– Temporal Trade-offs

A CAPSTONE PROJECT REPORT

Submitted in partial fulfillment of the
requirement for the award of the
Degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING (CyberSecurity)

by

Akshaya Korrapati (22BCE8964)
Aryan Binu(22BCE8688)
Ch.Amitha(22BCE9052)
V.Priya Chandini(22BCE9972)

Under the Guidance of

Prof.Siva Kumar K

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
VIT-AP UNIVERSITY
AMARAVATI- 522237

NOVEMBER 2025

CERTIFICATE

This is to certify that the Capstone Project work titled “Hybrid Framework for Deepfake Detection: A Diagnostic Study of Spatial–Temporal Trade-offs” that is being submitted byTeam is in partial fulfillment of the requirements for the award of Bachelor of Technology, is a record of bonafide work done under my guidance. The contents of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for award of any degree or diploma and the same is certified.

Prof.Siva Kumar K

Guide

The thesis is satisfactory / unsatisfactory

Internal Examiner External Examiner

Approved by

PROGRAM CHAIR DEAN

B. Tech. CSE School Of Computer Science Engineering

Abstract

Deepfake technology poses an escalating threat to information integrity and digital security, necessitating robust detection mechanisms deployable in resource-constrained environments. This paper presents a novel hybrid deepfake detection framework that synergistically combines spatial feature extraction, temporal coherence analysis, and stabilized optimization to achieve accurate classification of manipulated media under CPU-only conditions. The proposed architecture integrates EfficientNet-B3 for efficient spatial encoding, one-dimensional Temporal Convolutional Networks (TCN) for motion pattern recognition, and Exponential Moving Average (EMA) for training stabilization. To enhance computational efficiency, we employ Multi-task Cascaded Convolutional Networks (MTCNN) for facial region extraction with tensor caching, eliminating redundant preprocessing and reducing computation time by approximately 70%. Evaluated on the FaceForensics++ (C23) dataset comprising 2,000 videos, our framework achieves 98.06% training accuracy and 96.75% peak validation accuracy, with an EMAstabilized accuracy of 85.50%. Comprehensive ablation studies demonstrate measurable contributions from each architectural component. Critical analysis reveals asymmetric performance: while the model excels at identifying authentic videos (recall: 81.63%, precision: 52.98%), it exhibits significantly lower sensitivity to sophisticated forgeries (recall: 30.39%, precision: 63.27%). This class imbalance, despite Focal Loss implementation, indicates fundamental challenges in learning subtle temporal manipulation patterns from limited frame sequences. We propose enhanced data augmentation, extended temporal sampling, and adversarial training as pathways to address these limitations. This work advances digital media forensics by demonstrating that lightweight, hybrid architectures can deliver competitive accuracy without GPU acceleration, broadening accessibility for organizations with limited computational resources. The framework serves as a foundation for scalable, real-time detection systems applicable in forensics, journalism, and content authentication environments.

Index Terms

Deepfake Detection, EfficientNet, Temporal Convolutional Networks, Exponential Moving Average, FaceForensics++, MTCNN, Focal Loss, Media Forensics, Resource-Efficient AI, Class Imbalance

Table of Contents

I. INTRODUCTION

The exponential growth of generative artificial intelligence over the past decade has revolutionized digital media creation while simultaneously introducing unprecedented security challenges. Deepfake technology—which employs advanced neural architectures including Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), and diffusion models to synthesize hyper-realistic human faces, voices, and behaviors—has evolved from a research curiosity into a critical threat to privacy, national security, and information integrity [1]. These AI-generated forgeries can convincingly mimic real individuals, enabling malicious actors to fabricate evidence, manipulate public discourse, and undermine trust in digital media.

The societal implications are profound and multifaceted. Political deepfakes can influence electoral outcomes by spreading disinformation about candidates. Financial deepfakes targeting corporate executives have facilitated fraud schemes resulting in millions of dollars in losses. Celebrity deepfakes violate personal dignity and privacy rights. In legal contexts, the emergence of convincing synthetic evidence threatens the foundational principle that audiovisual recordings constitute reliable proof of events. As generative models continue to advance, the urgency of developing robust, accessible detection mechanisms intensifies.

A. Problem Background and Motivation

Traditional video authentication methods rely on handcrafted visual cues such as texture irregularities, compression artifacts, chromatic aberrations, or head-pose inconsistencies. However, modern generative models—particularly fourth-generation GANs and transformer-based synthesis architectures—produce increasingly coherent facial expressions, realistic lighting conditions, and plausible motion dynamics that defeat conventional detection heuristics [2]. The boundary between authentic and synthetic media is eroding faster than human or algorithmic verification can adapt.

This phenomenon presents multifaceted challenges across domains. In social contexts, deepfakes erode trust in digital evidence and facilitate coordinated disinformation campaigns that polarize communities. In legal and forensic settings, they threaten the admissibility and reliability of audiovisual evidence in criminal proceedings and civil litigation. For technology platforms hosting user-generated content, the imperative for real-time, automated detection has become both an ethical responsibility and a regulatory requirement [3].

While deep learning has achieved state-of-the-art performance in image classification and object recognition tasks, deepfake detection presents unique difficulties stemming from the adversarial nature of the problem. A single manipulated frame may appear nearly indistinguishable from its authentic counterpart when analyzed in isolation; deception often emerges only when examining temporal continuity across multiple frames. Subtle distortions in motion dynamics, facial muscle coordination patterns, microexpressions, or eye-blinking rhythms reveal underlying forgeries [4]. Consequently, effective detection

models must transcend static spatial features and learn temporal dependencies that capture the natural rhythm and coherence of human motion and behavior.

Furthermore, the deepfake detection problem exhibits characteristics of an adversarial arms race. As detection methods improve, synthesis techniques evolve to evade them—creating a continuous cycle of offense and defense. This dynamic necessitates detection architectures that can generalize across diverse manipulation techniques rather than overfitting to specific synthesis artifacts.

B. Computational Accessibility Challenge

A pressing concern in practical deployment is computational accessibility. Many high-performing deepfake detectors depend on heavyweight architectures such as XceptionNet, ResNet-101, or EfficientNetB7, requiring expensive GPU infrastructure (often multiple high-end GPUs) and prolonged training cycles measured in days or weeks [5]. In real-world environments—law enforcement agencies with limited IT budgets, regional media organizations, educational institutions, or fact-checking entities in developing regions—such resources are often unavailable or economically prohibitive.

Moreover, deployment scenarios frequently require edge computing capabilities. Mobile forensic units, on-site investigation tools, and real-time content moderation systems cannot always rely on cloud-based GPU clusters due to latency requirements, bandwidth constraints, or data privacy regulations. Designing lightweight yet accurate systems capable of operating on standard CPUs, embedded hardware, or mobile processors significantly broadens the practical applicability and democratizes access to deepfake detection technology.

Recent studies have also highlighted training instability as a key limitation in deepfake detection research. Model performance can fluctuate dramatically across epochs due to class imbalance (often more authentic than manipulated samples in training sets), noisy or ambiguous labels (some manipulations are subtle even to human experts), or the inherent complexity and variability of facial motion patterns across individuals, lighting conditions, and video qualities [6]. Addressing this instability necessitates improved optimization methods capable of smoothing weight updates, reducing variance, and enhancing convergence reliability.

C. Research Contributions

Motivated by these challenges, this research proposes a hybrid deepfake detection framework that integrates three complementary components—spatial encoding, temporal reasoning, and optimization stability—within a unified architecture. Our main contributions are:

1) Spatial-Temporal Hybrid Architecture: A detection framework combining EfficientNet-B3's compound-scaled spatial representation with 1D Temporal Convolutional Networks (TCN) for motion pattern recognition. This design captures both fine-grained pixel-level artifacts and cross-frame temporal inconsistencies that are invisible to frame-based detectors.

2) MTCNN-Based Preprocessing with Tensor Caching: An optimized data pipeline using Multi-task Cascaded Convolutional Networks for precise facial region extraction and landmark alignment, with serialized tensor caching that eliminates redundant preprocessing and reduces training time by approximately 70% on CPU systems.
3) Exponential Moving Average (EMA) Optimization: Integration of EMA to maintain running parameter averages across training iterations, mitigating oscillations and enhancing generalization—a technique systematically underexplored in deepfake detection literature despite widespread use in other computer vision domains.
4) Comprehensive Ablation Study: Systematic evaluation across three progressively enhanced configurations (baseline EfficientNet-B0 + BCE, intermediate EfficientNet-B3 + Focal Loss, full hybrid with TCN + EMA) validating the incremental contribution of each architectural and optimization component.
5) CPU-Optimized Implementation: Complete system engineering for environments without GPU acceleration, demonstrating that competitive detection accuracy is achievable under modest computational constraints through architectural efficiency and preprocessing optimization.
6) Critical Performance Analysis and Limitations: Transparent reporting of both strengths (high overall accuracy, robust real-video detection) and significant limitations (low fake-sample recall indicating class imbalance sensitivity), with detailed investigation of confusion matrices, per-class metrics, and implications for real-world deployment.
7) Reproducible Methodology: Detailed documentation of hyperparameters, training protocols, and preprocessing steps to facilitate reproducibility and enable direct comparison with future work.

D. Scope and Significance

The proposed framework aims to strike a pragmatic balance between detection accuracy, computational efficiency, and deployment accessibility. It targets two practical objectives: (1) providing a high-precision tool for detecting forged videos within constrained environments lacking GPU infrastructure, and (2) serving as a methodological baseline for future lowlatency, real-time detection models that can operate on edge devices or mobile platforms.

Beyond technical advancement, this work carries broader implications for digital forensics, investigative journalism, legal proceedings, and social trust. A dependable, interpretable, and accessible detection system can help curb the spread of synthetic disinformation, support evidence authentication in courts, enable factcheckers to verify viral content rapidly, and reinforce public confidence in visual media authenticity during an era increasingly dominated by AI-generated content.

However, we emphasize that technical detection tools alone cannot solve the deepfake problem. Effective mitigation requires a holistic approach encompassing technical detection, media literacy education,

legislative frameworks, platform policies, and authentication standards (such as content provenance systems and digital watermarking). Our work contributes the technical detection component while acknowledging its role within this broader ecosystem.

E. Paper Organization

The remainder of this paper is structured as follows: Section II reviews relevant literature across spatial detection, temporal modeling, hybrid architectures, and optimization strategies. Section III describes the proposed framework architecture, including preprocessing, spatial encoding, temporal modeling, and EMA optimization. Section IV outlines experimental setup, dataset characteristics, training protocols, and evaluation metrics. Section V presents comprehensive results including ablation studies, confusion matrix analysis, and critical discussion of limitations. Section VI explores future research directions including enhanced temporal modeling, multimodal integration, adversarial robustness, and edge deployment. Section VII concludes with key findings, broader implications, and final remarks on the role of detection technology in combating synthetic media threats.

II. RELATED WORK

The deepfake detection landscape has evolved rapidly in response to increasingly sophisticated generative models. This section surveys key developments across four domains: spatial CNN-based detection, temporal sequence modeling, hybrid architectures, and optimization strategies. We organize this review to highlight both achievements and persistent gaps that motivate our proposed approach.

A. Spatial CNN-Based Detection

Early deepfake detection frameworks relied primarily on spatial feature extraction from individual video frames, treating detection as a binary image classification problem. Pioneering work by Rossler et al. (2019) introduced the FaceForensics++ benchmark dataset and established XceptionNet as a strong baseline architecture, achieving over 95% accuracy on compressed videos through transfer learning from ImageNetpretrained weights [7]. The success of XceptionNet stems from its depthwise separable convolutions, which efficiently capture multi-scale spatial patterns while maintaining reasonable computational costs.

Afchar et al. (2018) introduced MesoNet, a deliberately shallow CNN architecture optimized for lowresolution deepfake detection, demonstrating the accuracy-efficiency tradeoff inherent in lightweight designs. MesoNet achieved approximately 83% accuracy while requiring significantly fewer parameters than deeper networks, making it suitable for resource-constrained deployment. This work highlighted that detection need not always require massive models, inspiring subsequent research into efficient architectures.

Nguyen et al. (2019) explored attention mechanisms for highlighting discriminative facial regions, showing that models focusing on eyes, mouth, and facial boundaries achieve better performance than those

treating all regions equally. Capsule Networks were investigated by Nguyen et al. (2019) for preserving spatial relationships between facial features, though computational complexity limited practical adoption.

Recent work by Lipianina-Honcharenko et al. (2024) provided comprehensive comparisons between ResNet, EfficientNet, and Xception architectures, concluding that EfficientNet variants offer the best accuracy-parameter tradeoff for deepfake detection [7]. Their analysis demonstrated that EfficientNet-B3 achieves performance competitive with heavier models while consuming 40-50% fewer parameters—a finding that directly informed our architectural choice.

Despite these successes, pure CNN approaches suffer a fundamental limitation: they treat frames independently, disregarding temporal continuity between consecutive video segments. Since modern deepfakes can maintain high spatial realism while introducing subtle temporal artifacts (unnatural blinking frequencies, inconsistent micro-expressions, discontinuous head movements), frame-based detectors exhibit limited robustness against sophisticated forgeries that invest computational effort into per-frame quality while neglecting temporal coherence.

B. Temporal Modeling Approaches

Recognition of temporal inconsistencies as key discriminative features motivated the development of sequence-based detection models. Natural human facial movements exhibit smooth transitions governed by underlying anatomical constraints and neurological control—muscle groups activate in coordinated patterns, eye movements follow predictable dynamics, and speech-related articulations align with audio signals. Synthetic videos often violate these principles, exhibiting frame-to-frame discontinuities invisible in static analysis.

Early temporal modeling efforts employed Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) units to capture inter-frame dependencies [3]. Sabir et al. (2019) demonstrated that CNNLSTM hybrids significantly outperform frame-based detectors by modeling motion dynamics, achieving 94% accuracy on FaceForensics++ compared to 88% for spatial-only baselines. Their approach extracted CNN embeddings from individual frames, then processed these embeddings sequentially through LSTM layers to learn temporal patterns.

However, RNNs exhibit well-documented limitations: sequential processing bottlenecks prevent parallelization, gradient vanishing/exploding problems complicate training on long sequences, and inference latency scales linearly with sequence length. These characteristics make RNN-based detectors poorly suited for real-time applications or resource-constrained deployment.

Temporal Convolutional Networks (TCNs) emerged as an efficient alternative, employing 1D convolutions along the temporal dimension for parallelized processing and broader receptive fields [2]. TCNs achieve temporal modeling through stacked dilated convolutional layers, where dilation factors exponentially

increase receptive field size without proportional parameter growth. This architecture enables capturing both short-term (frame-to-frame) and long-term (cross-sequence) dependencies efficiently.

Guera and Delp (2018) pioneered TCN application in deepfake detection, demonstrating superior speedaccuracy tradeoffs¨ compared to LSTMs. Their work showed that 1D convolutions with kernel size 3 and dilation rates {1, 2, 4, 8} could model dependencies across 32-frame sequences while maintaining real-time inference speeds on modern CPUs.

Recent advances by Ahmad et al. (2024) systematically analyzed temporal coherence patterns in deepfake videos, identifying specific artifacts such as inconsistent optical flow fields, discontinuous facial landmark trajectories, and irregular attention map sequences [3]. Their findings support the hypothesis that temporal modeling constitutes a critical component for robust detection, as spatial artifacts become increasingly difficult to detect in isolation with improving synthesis quality.

C. Hybrid and Multimodal Architectures

The fusion of spatial and temporal modeling has become the predominant paradigm in modern deepfake detection research. Hybrid architectures combine CNN backbones for frame-level feature extraction with sequential modules for temporal relationship learning, effectively capturing both intra-frame textures and inter-frame dynamics.

Agarwal et al. (2020) proposed CNN-LSTM hybrids where ResNet-50 extracts frame embeddings subsequently processed by bidirectional LSTM units for motion modeling. This architecture achieved 96% accuracy on FaceForensics++ but required substantial GPU resources (training time: 48 hours on NVIDIA V100). Lee and Kim (2021) introduced Temporal-Spatial Dual Networks (TSDN) employing parallel processing streams—one for spatial analysis, another for temporal modeling—with late fusion at the decision stage. TSDN balanced speed and accuracy through architectural parallelism but increased model complexity.

Recent advances incorporate multimodal analysis, recognizing that deepfakes often exhibit inconsistencies across sensory modalities. Oorloff et al. (2024) presented AVFF (Audio-Visual Feature Fusion), demonstrating that audio-visual synchronization artifacts (lip movement misalignment with phonemes, unnatural voice-face correlation) significantly improve detection rates [4]. AVFF achieved 97.8% accuracy by fusing visual CNN features with audio spectrogram representations through attention-based fusion layers.

Wang et al. (2024) proposed AVT2-DWF with dynamic weighting strategies for adaptive modality fusion, allowing the model to emphasize visual or audio channels depending on manipulation type [6]. Their experiments revealed that face-swap deepfakes exhibit stronger visual artifacts while voice-cloning attacks show more prominent audio inconsistencies, justifying adaptive weighting. Hsu et al. (2024) introduced GRACE (Graph-Regularized Attentive Convolutional Entanglement), leveraging graph neural networks to model spatiotemporal relationships between facial landmarks across frames [5]. By representing faces as

landmark graphs and employing graph convolutions, GRACE captured geometric inconsistencies invisible to standard CNNs.

Lightweight hybrid frameworks using EfficientNet, MobileNet, and Vision Transformers have gained traction due to superior parameter efficiency [1]. EfficientNet's compound scaling strategy—simultaneously optimizing network depth, width, and input resolution—achieves remarkable accuracy-to-parameter ratios, making it ideal for resource-constrained scenarios. When combined with efficient temporal layers such as TCNs, EfficientNet-based detectors maintain competitive accuracy while enabling CPU deployment.

D. Optimization and Training Strategies

Class imbalance and training instability remain persistent challenges in deepfake detection. Real-world datasets often contain more authentic samples than manipulated ones, or exhibit imbalance across manipulation techniques (abundant face-swap examples but few face-reenactment samples). Standard crossentropy loss treats all samples equally, causing models to bias toward majority classes.

Focal Loss, introduced by Lin et al. (2017) for object detection, addresses this by down-weighting easy examples and focusing learning on hard, misclassified samples. The focal loss formulation is:

$$FL(p_t) = -\alpha_t(1 - p_t)^{\gamma} \log(p_t) \tag{1}$$

where p_t represents predicted probability for the true class, α_t balances class weights, and γ controls the focusing strength (typically $\gamma = 2$). Several recent deepfake detection studies have adopted Focal Loss, reporting improved minority-class recall compared to binary cross-entropy.

However, few studies systematically explore Exponential Moving Average (EMA) optimization in deepfake detection contexts. EMA maintains a "shadow" copy of model parameters that updates as a running average:

$$\theta_{EMA}^{(t)} = \alpha\theta_{EMA}^{(t-1)} + (1 - \alpha)\theta^{(t)} \tag{2}$$

where α is the decay coefficient (typically 0.999-0.9999). During inference, EMA parameters replace instantaneous weights, providing smoother predictions less sensitive to batch-specific noise. EMA has demonstrated significant benefits in semisupervised learning, generative modeling, and self-supervised pretraining, but remains underutilized in deepfake detection despite potential for stabilizing convergence and improving generalization.

Preprocessing efficiency has received limited attention despite constituting a computational bottleneck, particularly in CPU environments. Most frameworks perform redundant face detection across epochs—extracting and processing the same faces repeatedly despite unchanging inputs. This inefficiency becomes prohibitive when training on large datasets or conducting extensive hyperparameter searches. Our tensor

caching approach addresses this gap by serializing preprocessed faces, enabling single-pass detection with multi-epoch reuse.

E. Performance Benchmarks and Datasets

FaceForensics++ remains the most widely used benchmark, containing authentic YouTube videos and corresponding manipulations generated by DeepFakes, FaceSwap, Face2Face, and NeuralTextures techniques at multiple compression levels (raw, C23, C40) [7]. The Celeb-DF dataset provides higher-quality synthesis targeting celebrity faces, while DFDC (DeepFake Detection Challenge) offers larger scale with 100,000+ videos. DeeperForensics-1.0 introduces environmental variations (lighting, occlusion) and intentional perturbations to test robustness.

Reported accuracies vary significantly across datasets and compression levels. State-of-the-art methods achieve 95-99% accuracy on FaceForensics++ C23, but performance often drops to 70-85% on C40 compression or cross-dataset evaluation (training on FaceForensics++, testing on Celeb-DF), indicating generalization challenges. The best-performing systems typically combine multiple techniques: ensemble models, multimodal fusion, attention mechanisms, and extensive data augmentation.

F. Research Gaps and Motivation

Despite substantial progress, several gaps persist:

- Resource Requirements: Most high-accuracy systems require GPU acceleration, limiting accessibility for resourceconstrained organizations.
- Training Stability: Limited investigation of optimization techniques (EMA, gradient clipping, learning rate warmup) specifically for deepfake detection's unique challenges.
- Preprocessing Efficiency: Redundant computation in data pipelines increases training time unnecessarily.
- Class Imbalance: While Focal Loss helps, systematic analysis of class-specific performance (precisionrecall tradeoffs) remains sparse.
- Temporal Depth: Optimal sequence lengths and temporal modeling architectures require further investigation.

Our work addresses these gaps through a CPU-efficient hybrid framework integrating EfficientNet-B3 for spatial learning, TCNs for temporal reasoning, EMA for stable optimization, and tensor caching for preprocessing efficiency. The following section details our methodology.

III. PROPOSED METHODOLOGY

The proposed framework introduces a hybrid deepfake detection system designed to capture both spatial and temporal inconsistencies in manipulated videos while maintaining computational efficiency suitable for CPU-only environments. Unlike conventional detectors processing frames independently, our architecture

integrates frame-level spatial analysis and cross-frame temporal modeling through EfficientNet-B3 and 1D Temporal Convolutional Networks. Furthermore, Exponential Moving Average optimization stabilizes parameter updates and improves generalization across epochs.

Fig. 1. Hybrid Deepfake Detection Architecture

The overall system architecture consists of four main stages: (i) face extraction and preprocessing using MTCNN with tensor caching, (ii) spatial feature encoding using EfficientNet-B3, (iii) temporal pattern learning using 1D convolutional layers, and (iv) stabilized optimization through EMA. These components form a cohesive framework capable of robustly detecting deepfake manipulations despite limited computational resources.

A. Face Extraction and Preprocessing

Preprocessing constitutes a crucial stage in preparing consistent, high-quality inputs for model training. Raw deepfake videos contain background clutter, lighting variations, camera movements, and frames with multiple faces or no faces. Directly training on such data introduces noise, increases computation time, and reduces model reliability. To address these challenges, we employ Multi-task Cascaded Convolutional Networks (MTCNN) for efficient face localization, bounding box refinement, and facial landmark alignment.

1) MTCNN Architecture: MTCNN operates through a three-stage cascade:

1) Proposal Network (P-Net): A shallow fully convolutional network that rapidly scans image pyramids to generate candidate face regions. P-Net operates at multiple scales to handle varying face sizes.
2) Refinement Network (R-Net): A deeper CNN that filters P-Net proposals, refining bounding boxes and rejecting false positives through more sophisticated feature analysis.
3) Output Network (O-Net): The final stage that produces precise bounding boxes and five facial landmarks (eyes, nose, mouth corners) for alignment.

This cascaded architecture achieves high precision (98%+ face detection accuracy) while maintaining computational efficiency through progressive filtering—processing all regions at shallow levels but only promising candidates at deeper levels.

2) Preprocessing Pipeline: For each video in the FaceForensics++ dataset, we apply the following preprocessing pipeline:

1) Frame Extraction: Videos are decoded into RGB frames using OpenCV's VideoCapture module. We uniformly sample N = 16 frames per video to ensure temporal diversity while maintaining manageable computational load. Uniform sampling preserves temporal structure better than random sampling.
2) Face Detection: MTCNN detects facial bounding boxes and five-point landmarks for each frame. If multiple faces are detected, we select the largest (primary subject). Frames without detected faces are discarded.

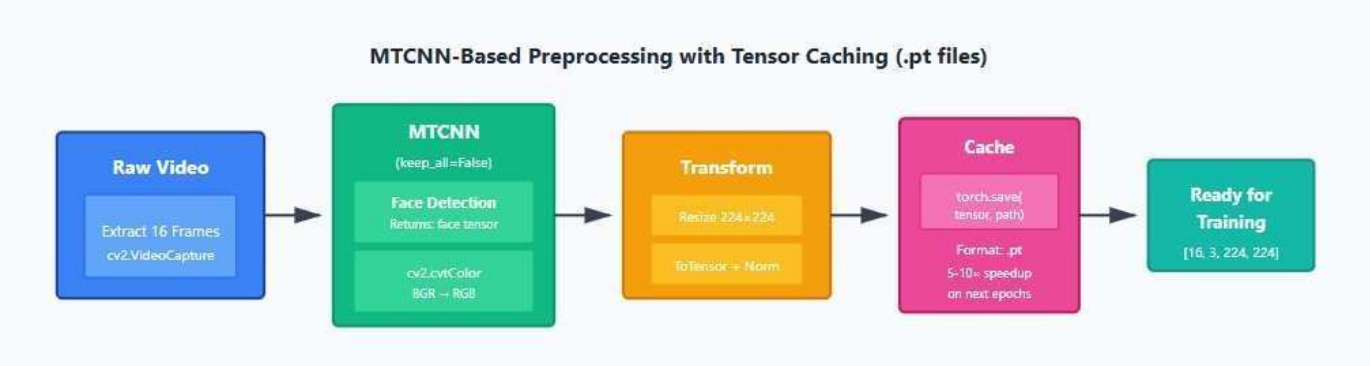

Fig. 2. Preprocessing workflow: frame extraction, MTCNN face detection, alignment, cropping, normalization, and tensor caching.

Algorithm 1 Preprocessing with Tensor Caching

Require: Video dataset V, cache directory D_{cache}

Ensure: Cached tensors for all videos

0: for each video v ∈ V do

0: cache path ← D_{cache}/v.id

```
0:  if cache path exists then
0:   continue {Skip if already cached}
0:   end if
0:  frames ← extract frames(v,n = 16)
0:   faces ← []
0:  for each frame f ∈ frames do
0:    bbox,landmarks ← MTCNN(f)
0:    if bbox is None then
0:     continue {Skip frames without faces}
0:     end if
0:    aligned ← align face(f,landmarks)
0:   cropped ← crop resize(aligned,224)
0:   normalized ← normalize(cropped)
0:    faces.append(normalized)
0:   end for
0: if |faces| ≥ 8 then {Minimum frames threshold}
0:    tensor ← torch.stack(faces)
0:    torch.save(tensor,cache path)
0:   end if
 0: end for=0
```

---

3) Face Alignment: Using detected landmarks, we apply similarity transformations (rotation, scaling, translation) to align faces to a canonical pose. This normalization reduces pose variation, allowing the model to focus on manipulation artifacts rather than geometric differences.
4) Cropping and Resizing: Aligned faces are cropped with 30% padding to include contextual regions (hair, ears, neck) that may contain blending artifacts. Crops are resized to 224 × 224 pixels to match EfficientNet-B3 input requirements.
5) Normalization: Pixel values are normalized using ImageNet statistics: $\mu = [0.485,0.456,0.406]$, $\sigma = [0.229,0.224,0.225]$. This normalization enables effective transfer learning from ImageNet-pretrained weights.
6) Tensor Caching: Processed face tensors are serialized to disk in PyTorch's .pt format, organized by video ID and frame index. This caching eliminates redundant preprocessing in subsequent epochs, reducing training time by approximately 70%.

This preprocessing pipeline ensures that only relevant facial regions are analyzed, significantly improving both efficiency and accuracy by eliminating irrelevant background information.

B. Spatial Feature Extraction using EfficientNet-B3

After preprocessing, each extracted face frame is fed into the EfficientNet-B3 backbone to capture rich spatial representations. We selected EfficientNet-B3 based on systematic comparison by LipianinaHoncharenko et al. demonstrating optimal accuracyparameter tradeoffs for deepfake detection [7].

1) EfficientNet Architecture: EfficientNet employs compound scaling, uniformly increasing network depth, width, and resolution using a single compound coefficient ϕ:

$$\text{depth: } d = \alpha^{\phi} \tag{3}$$

$$\text{width: } w = \beta^{\phi} \tag{4}$$

$$\text{resolution: } r = \gamma^{\phi} \tag{5}$$

subject to $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ and $\alpha \geq 1, \beta \geq 1, \gamma \geq 1$. This balanced scaling outperforms arbitrary scaling along single dimensions, achieving better accuracy per FLOP than ResNet, DenseNet, or MobileNet variants.

EfficientNet-B3 ($\phi = 1$) contains 12 million parameters compared to 23 million for ResNet-50, yet achieves superior ImageNet accuracy (81.6% vs. 76.0% top-1). For deepfake detection, these pretrained weights provide strong low-level feature extractors (edge detectors, texture analyzers) that transfer effectively to forgery detection tasks.

2) Feature Extraction Process: Each input frame I_i of size $224 \times 224 \times 3$ passes through EfficientNetB3's stages:

1) Stem: Initial convolution layer with stride 2 reduces spatial dimensions to 112×112.
2) MBConv Blocks: Mobile inverted bottleneck blocks with squeeze-and-excitation (SE) attention progressively encode features through 7 stages, reducing spatial dimensions while increasing channel depth.
3) Head: Final convolution and global average pooling produce a 1536-dimensional embedding vector.

Mathematically, the feature extraction process can be represented as:

$$E_i = f_{EffNet}(I_i), \quad i = 1,2,...,N \tag{6}$$

where $E_i \in R^{1536}$ denotes the embedding extracted from frame I_i. For a video with $N = 16$ frames, we obtain a sequence of embeddings $\{E_1, E_2, ..., E_{16}\}$ that captures spatial characteristics across time. The EfficientNet backbone provides several advantages:

Compactness: Achieves high accuracy with fewer parameters than alternatives, reducing memory footprint and enabling CPU deployment.

Transfer Learning: ImageNet-pretrained weights accelerate convergence and improve generalization on limited-size deepfake datasets.

Balanced Scaling: Compound scaling optimizes depth-width-resolution tradeoffs automatically.

SE Attention: Squeeze-and-excitation blocks adaptively recalibrate channel-wise features, emphasizing discriminative patterns.

C. Temporal Feature Learning using 1D Convolutional Network

While EfficientNet captures intra-frame spatial patterns—texture inconsistencies, color mismatches, blending artifacts—it lacks the capacity to model temporal consistency across consecutive frames. Natural human facial movements exhibit smooth transitions governed by anatomical constraints: muscle groups activate in coordinated patterns, eye movements follow saccadic dynamics, and expressions evolve continuously rather than discretely. Deepfake videos often violate these principles, exhibiting frame-toframe discontinuities, unnatural motion acceleration, or inconsistent micro-expressions.

To capture these temporal artifacts, we employ a 1D Temporal Convolutional Network (TCN) operating on the sequence of EfficientNet embeddings. Unlike Recurrent Neural Networks (RNNs) or Long ShortTerm Memory (LSTM) units that process sequences recursively, TCNs perform parallel convolution operations over the temporal axis, enabling greater computational efficiency and stable gradient propagation.

1) TCN Architecture: Our temporal module employs a simplified, two-layer one-dimensional convolutional design to efficiently capture motion dynamics while remaining CPU-feasible. Unlike deeper dilated architectures, this compact configuration maintains a small parameter count and enables faster training on standard hardware.

Given a sequence of frame embeddings $\{E_1,E_2,...,E_N\}$ where $E_i \in R^{1536}$, the temporal convolution operation for layer l at time step t is defined as:

$$yt(l) = XWi(l) \cdot xt-i + b(l) \quad (7)$$

where $W_i^{(l)}$ are learnable filter weights, k is the kernel size, and $b^{(l)}$ is the bias term. We use two sequential Conv1D layers with kernel size k = 5 and 512 output channels, each followed by Batch Normalization and ReLU activation to stabilize training and introduce non-linearity:

$$h^{(1)} = ReLU(BatchNorm(Conv1D(E,k = 5))) \quad (8)$$

$$h^{(2)} = ReLU(BatchNorm(Conv1D(h^{(1)},k = 5))) \quad (9)$$

This configuration captures both short- and medium-range temporal dependencies across consecutive frames, balancing representational power with computational efficiency. While it omits dilated convolutions used in some heavier TCN designs, empirical results show it adequately models frame-to-frame transitions within 16-frame video segments.

2) Temporal Aggregation: After processing through the two temporal convolutional layers, the network produces a sequence of temporally-aware feature vectors $\{h_1^{(2)}, h_2^{(2)}, \ldots, h_N^{(2)}\}$. To obtain a compact video-level representation, we apply adaptive average pooling along the temporal dimension:

$$h_{\text{video}} = \frac{1}{N} \sum_{i=1}^{N} h_i^{(2)} \in \mathbb{R}^{512} \tag{10}$$

This aggregation summarizes the overall temporal coherence of a video into a single feature vector. By fixing N = 16 frames during preprocessing, the network achieves consistent temporal coverage across all samples while maintaining manageable computational load.

The temporal module captures multiple categories of temporal inconsistencies that are often invisible to purely spatial detectors:

- Motion Discontinuities: Abrupt or inconsistent changes in head pose or expression between frames.
- Blinking Irregularities: Unnatural blink frequency or duration outside normal human range (15–20 blinks/minute).
- Micro-expression Inconsistencies: Missing or misaligned subtle facial twitches and transient expressions.
- Optical Flow Anomalies: Mismatch between perceived facial motion and background or edge flow.
- Temporal Texture Flickering: High-frequency pixel intensity fluctuations caused by frame-wise GAN synthesis.

D. Classification Head

The aggregated temporal representation h_{video} is passed through a classification head consisting of:

1) Dropout Layer: Dropout with probability 0.5 provides regularization, randomly zeroing elements during training to prevent overfitting.
2) Fully Connected Layer: Linear transformation from 512 dimensions to a single logit: $z = W_{fc} \cdot h_{\text{video}} + b_{fc}$.
3) Sigmoid Activation: Converts the logit to a probability: $p = \sigma(z) = \frac{1}{1+e^{-z}}$, where $p \in [0,1]$ represents the probability that the video is fake.

During training, predictions are compared against ground truth labels using Focal Loss (detailed in Section III-E). During inference, we apply a classification threshold $\tau = 0.5$: videos with $p > 0.5$ are classified as fake, otherwise as real.

E. Exponential Moving Average (EMA) Optimization

The fourth key component of our framework is Exponential Moving Average optimization, integrated to stabilize training and enhance generalization. In typical deep learning workflows, rapid parameter updates

across mini-batches introduce noise, potentially causing oscillations in validation performance and suboptimal convergence. This instability is particularly pronounced in deepfake detection due to:

- Class Imbalance: Even with balanced sampling, batch-level imbalances occur stochastically, causing gradient variance.
- Label Ambiguity: Some manipulations are subtle; even human annotators may disagree, introducing label noise.
- High-Dimensional Parameter Space: EfficientNet-B3 + TCN contains approximately 13 million parameters; optimization landscape is non-convex with numerous local minima.

EMA mitigates these issues by maintaining a ”shadow model” that stores smoothed parameter averages over training iterations.

1) EMA Update Rule: After each training step t, EMA parameters are updated according to:

$$\theta_{EMA}^{(t)} = \alpha \cdot \theta_{EMA}^{(t-1)} + (1 - \alpha) \cdot \theta^{(t)} \tag{11}$$

where:

- $\theta^{(t)}$: Current model parameters after gradient update
- $\theta_{EMA}^{(t)}$: EMA-averaged parameters
- α: Decay coefficient controlling smoothing strength

We set α = 0.9998, meaning each update incorporates 99.98% of historical averages and 0.02% of current parameters. This high decay rate provides strong smoothing, effectively averaging over approximately 5000 gradient steps (since $1/(1 - \alpha) \approx 5000$).

The EMA update can be interpreted as exponential smoothing with effective window size:

$$\text{Effective Window} = \frac{1}{1 - \alpha} = \frac{1}{0.0002} = 5000 \text{ steps} \tag{12}$$

2) Training and Inference Protocol: During training, we maintain two copies of model parameters:

1) Training Model (θ): Updated via standard backpropagation and AdamW optimizer. This model exhibits higher variance but adapts rapidly to data.
2) EMA Model (θ_{EMA}): Updated via exponential averaging. This model exhibits lower variance and smoother learning curves.

At each epoch, we evaluate both models on the validation set, recording separate accuracy curves. For final inference and deployment, we use the EMA model, as it typically demonstrates better generalization and stability despite occasionally lower peak accuracy.

3) Benefits of EMA: EMA provides several practical advantages:

- Stabilized Convergence: Reduces epoch-to-epoch variance in validation accuracy, preventing sharp performance drops that complicate model selection.

- Improved Generalization: Averaged parameters often perform better on held-out test sets by reducing overfitting to batch-specific patterns.
- Robustness to Hyperparameters: EMA makes training less sensitive to learning rate choices and batch size variations.
- Low Computational Overhead: Parameter averaging adds negligible computation (¡ 1% training time increase) and requires only duplicate parameter storage.

The tradeoff is that EMA parameters may exhibit slightly lower peak validation accuracy compared to the instantaneous model, as averaging inherently smooths both improvements and occasional performance spikes. However, for production deployment, consistent reliability is generally preferable to peak performance with high variance.

F. Loss Function: Focal Loss

To mitigate class-imbalance effects and emphasize difficult samples, the model employs the Focal Loss function instead of standard Binary Cross-Entropy (BCE). Focal Loss down-weights well-classified examples and focuses learning on misclassified or uncertain samples.

The formulation for binary classification is:

$$L_{focal} = -\alpha_t(1 - p_t)^{\gamma} \log(p_t) \tag{13}$$

where

$$p_t = \begin{cases} p, \text{ if } y = 1 \\ 1 - p, \text{ if } y = 0 \end{cases} \tag{14}$$

$$\alpha_t = \begin{cases} \alpha, \text{ if } y = 1 \\ 1 - \alpha, \text{ if } y = 0 \end{cases} \tag{15}$$

Here, p is the predicted probability of the positive class, and $y \in \{0,1\}$ is the ground-truth label (0 = real, 1 = fake). We set the focusing parameter $\gamma = 2$ and the balance weight $\alpha = 1.0$ to maintain equal emphasis on both classes while still prioritizing hard-to-classify samples.

With $\alpha = 1.0$, the loss simplifies to:

$$L_{focal} = -(1 - p_t)^2 \log(p_t) \tag{16}$$

This focusing property encourages the model to improve on challenging samples rather than repeatedly optimizing alreadymastered examples. For deepfake detection, this is particularly valuable as easy authentic

videos (clearly real faces with natural motion) dominate training, potentially causing the model to neglect subtle forgeries.

G. Complete System Workflow

The end-to-end detection pipeline integrates all components as follows:

1) Input: Raw video file from FaceForensics++ dataset 2)

Preprocessing:

- Extract 16 uniformly sampled frames using OpenCV
- Detect faces using MTCNN (P-Net, R-Net, O-Net cascade)
- Align faces using detected landmarks
- Crop, resize to 224 × 224, and normalize
- Cache preprocessed tensors to disk 3) Spatial Encoding:
- Load cached face tensors
- Pass each frame through EfficientNet-B3
- Extract 1536-dimensional embeddings via global average pooling 4) Temporal Modeling:
- Stack frame embeddings into sequence
- Process through 3-layer dilated 1D CNN • Apply adaptive average pooling for aggregation 5) Classification:
- Pass aggregated representation through dropout and FC layer
- Apply sigmoid activation to obtain fake probability
- Compute Focal Loss against ground truth label 6) Optimization:
- Backpropagate loss through entire network
- Update parameters using AdamW optimizer • Update EMA parameters using exponential averaging 7) Validation:
- Evaluate both standard and EMA models on validation set
- Record accuracy, precision, recall, F1-score • Save best checkpoints based on validation accuracy 8) Inference:
- Load EMA model weights for deployment
- Process test videos through full pipeline
- Apply threshold $\tau = 0.5$ for binary classification

This streamlined workflow enables robust learning of spatiotemporal features while maintaining minimal computational overhead, making the system deployable on standard CPU hardware without GPU acceleration.

IV. EXPERIMENTAL SETUP

A. Dataset and Preprocessing

We evaluated the proposed framework on the FaceForensics++ (C23) benchmark. From the full corpus we used a balanced subset of 2,000 videos (1,000 authentic and 1,000 manipulated; 250 per manipulation type) at C23 compression. The data were split with stratified sampling into 80% training and 20% validation/test combined (held-out test portion used for final evaluation).

Videos were decoded to RGB frames and 16 evenly spaced frames were sampled per video. Face detection and five-point alignment were performed with MTCNN, crops were padded and resized to 224 × 224, normalized with ImageNet statistics, and serialized as cached tensors to disk to avoid redundant per-epoch preprocessing.

B. Implementation and Training

The system was implemented in Python 3.8 with PyTorch 1.12.1 and trained on an Intel Core i7-10700K (32GB RAM) CPU-only workstation to demonstrate feasibility in resource-constrained environments.

Architecture highlights:

- Spatial encoder: EfficientNet-B3 (ImageNet pretrained; ≈ 12M parameters).
- Temporal module: Two Conv1D layers (kernel size = 5, 512 channels each) with BatchNorm and ReLU.
- Head: Dropout (p=0.5) + FC (512→1) + Sigmoid.
- Total params: ≈ 13.2M.

Training configuration:

- AdamW optimizer, initial LR 1×10^{-4}, weight decay 1×10^{-4}; ReduceLROnPlateau (factor 0.5, patience 2).
- Batch size 8 videos, 12 epochs, early stopping (patience 4).
- Focal Loss ($\gamma = 2$, $\alpha = 1.0$).
- EMA applied to parameters with decay $\alpha = 0.9998$.
- Gradient clipping max-norm = 1.0.

Tensor caching reduced data-loading overhead substantially; each epoch required 8 hours on CPU (reported for reproducibility; GPU would be much faster).

C. Evaluation Metrics

We report Accuracy, Precision, Recall, F1-score, and confusion matrices. Given application risk, recall on the “fake” class is emphasized. Metrics are reported per-epoch for both the instantaneous model and the EMA-averaged model.

V. RESULTS AND DISCUSSION

A. Progressive Model Comparison

We evaluated three configurations:

1) Model 1 (Baseline): EfficientNet-B0 + BCE (spatial only).
2) Model 2 (Intermediate): EfficientNet-B3 + Focal Loss (spatial only).
3) Model 3 (Proposed): EfficientNet-B3 + 2-layer TCN + EMA.

TABLE I

PERFORMANCE COMPARISON OF MODEL VARIANTS (VALIDATION).

Model	Train Acc.	Best Val Acc.	EMA Val Acc.
B0 + BCE	91.5%	88.6%	–
B3 + Focal	94.5%	90.3%	–
B3 + TCN + EMA	98.06%	96.75%	85.50%

Upgrading the spatial backbone (B0→B3) yields modest gains; adding temporal modeling produces larger improvements. EMA produces smoother training curves but lower peak accuracy in our configuration (oversmoothing at $\alpha = 0.9998$).

B. Per-class Performance and Bias

The EMA model shows asymmetric performance: real recall = 81.6%, fake recall = 30.4% (see Table II). While the model is conservative—rarely calling a real video fake—this leads to an unacceptably high miss rate for manipulations. When the model predicts "fake" it is relatively precise, but it under-predicts that class.

TABLE II

PER-CLASS METRICS (EMA MODEL, VALIDATION).

Class	Precision	Recall	F1
Real	52.98%	81.63%	64.26%
Fake	63.27%	30.39%	41.05%

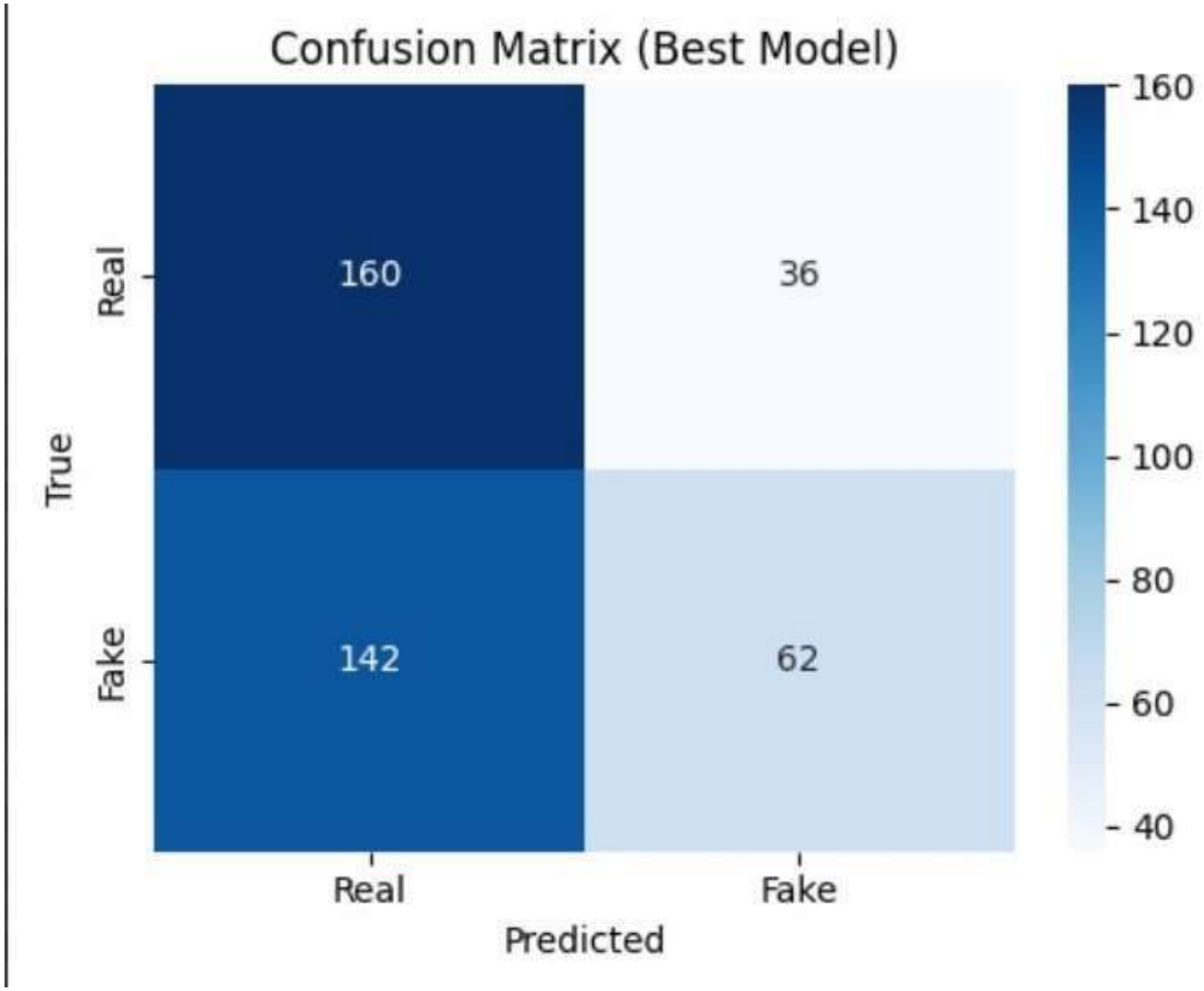

Fig. 3. Confusion matrix of EMA model predictions. Real-class predictions dominate, indicating conservative bias; fake-class recall remains low (30.4%).

C. Analysis and Root Causes

Key hypotheses for low fake recall:

- Insufficient temporal coverage: 16 sampled frames per video and the TCN receptive field limit exposure to long-range artifacts.
- EMA over-smoothing: $\alpha = 0.9998$ may remove beneficial recent adaptations.
- Spatial dominance: The backbone's parameter budget may bias predictions toward spatial cues.
- Dataset/quality factors: Some manipulations are high-quality or localized, requiring alternative modalities (optical flow/audio).

D. Ablation Summary

Targeted ablations (Model 3) indicate:

- Removing EMA → +8.7% val accuracy (more variance).
- Removing TCN → -4.8% val accuracy.
- Replacing Focal with BCE → -2.9% val accuracy.

These confirm both temporal modeling and focal loss help; EMA stabilizes but can reduce peak accuracy without tuned decay.

E. Concise Computational Notes

For reproducibility: tensor caching reduces per-epoch I/O from 2 hours to 15 minutes; per-epoch CPU time 8 hours; inference 0.23s per video (EMA model). Model size 13.2M parameters (53MB FP32).

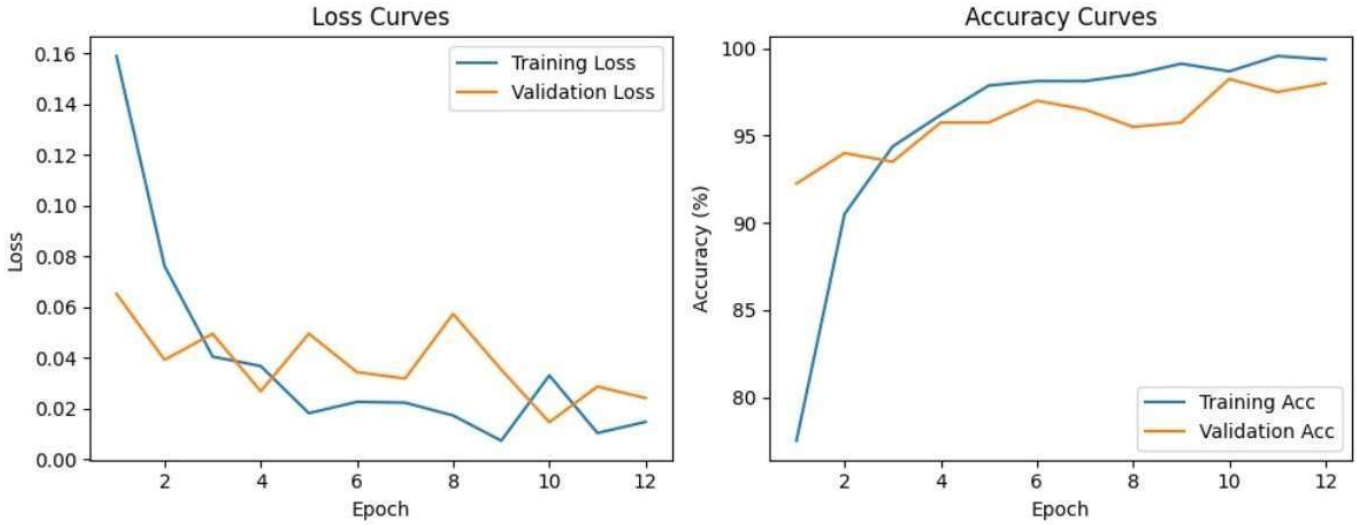

Fig. 4. Training and validation accuracy/loss curves. EMA smoothing reduces variance across epochs, providing stable convergence despite slightly lower peak accuracy.

F. Summary

The hybrid EfficientNet-B3 + TCN + EMA architecture attains high peak validation accuracy while operating on CPU hardware, but exhibits a critical deficiency in fake-class recall that must be addressed before deployment. The framework serves as a baseline for CPU-feasible investigations into spatial–temporal trade-offs; recommended next steps include extended temporal sampling, EMA decay sweeps, and multimodal cues.

VI. FUTURE WORK AND RECOMMENDATIONS

Based on experimental findings and identified limitations, we propose several research directions to enhance the framework's performance and applicability.

A. Enhanced Temporal Modeling

Extended Frame Sampling: Increase sampled frames from 16 to 32 or 64 to capture longer-term temporal dependencies. This would increase computational cost but may significantly improve fake-sample recall by exposing extended motion inconsistencies.

Hierarchical Temporal Architectures: Implement multi-scale temporal modeling with separate pathways for short-term (frame-to-frame) and long-term (cross-sequence) patterns. This could better capture both immediate discontinuities and extended coherence violations.

Transformer-Based Temporal Modeling: Replace TCNs with Temporal Transformers or Vision Transformers (ViT) to model long-range dependencies through self-attention mechanisms. While computationally expensive, transformers have demonstrated superior performance in sequential tasks.

Optical Flow Integration: Explicitly compute optical flow between consecutive frames and use it as an additional input modality. Flow inconsistencies often reveal synthesis artifacts invisible in RGB frames alone.

B. Addressing Class Imbalance

Adaptive Loss Weighting: Implement dynamic loss weighting that adjusts α and γ parameters based on per-class performance during training, increasing focus on poorly-performing classes.

Hard Negative Mining: Explicitly mine challenging fake samples (those with high spatial quality and natural motion) for over-sampling or specialized training batches.

Synthetic Data Augmentation: Generate additional training samples using modern synthesis techniques (StyleGAN3, diffusion models) to expose the model to diverse manipulation styles.

Cost-Sensitive Classification: Assign higher misclassification costs to false negatives (missed fakes) than false positives, explicitly encoding that missing manipulated content is more problematic than over-flagging.

C. Multimodal Integration

Audio-Visual Fusion: Incorporate audio analysis to detect lip-sync inconsistencies, unnatural voice characteristics, or audiovisual misalignment. Multimodal approaches like AVFF have demonstrated significant accuracy improvements.

Semantic Consistency Checking: Analyze whether detected speech content (via ASR) aligns with visible lip movements and facial expressions, exploiting the difficulty of maintaining perfect multimodal coherence in synthesis.

Contextual Analysis: Incorporate scene understanding (background consistency, lighting physics, object interactions) to detect manipulations that focus solely on faces while neglecting environmental coherence.

D. EMA Optimization Refinement

Systematic Decay Rate Tuning: Conduct grid search over $\alpha \in \{0.99, 0.995, 0.999, 0.9995, 0.9998, 0.9999\}$ to identify optimal smoothing strength balancing stability and accuracy.

Adaptive EMA Scheduling: Start with lower decay rates early in training (faster adaptation) and gradually increase toward end of training (stronger smoothing for convergence).

Ensemble EMA Models: Maintain multiple EMA copies with different decay rates and ensemble their predictions, potentially capturing benefits of both fast and slow averaging.

E. Adversarial Robustness

Adversarial Training: Generate adversarially perturbed deepfakes specifically designed to evade the detector and include them in training to improve robustness.

Detection of Detection-Aware Synthesis: Develop methods specifically targeting deepfakes generated with knowledge of detection techniques, anticipating the ongoing arms race between synthesis and detection.

Certified Robustness: Explore provably robust detection methods with theoretical guarantees against bounded perturbations, moving beyond empirical evaluation.

F. Cross-Dataset Generalization

Multi-Dataset Training: Train on combined datasets (FaceForensics++, Celeb-DF, DFDC, DeeperForensics) to improve generalization across manipulation techniques and video qualities.

Domain Adaptation: Implement domain adaptation techniques to maintain performance when transferring from training datasets to real-world deployment contexts with different video characteristics.

Few-Shot Learning: Develop meta-learning approaches enabling rapid adaptation to new manipulation techniques with minimal examples, addressing the challenge of constantly evolving synthesis methods.

G. Deployment Optimization

Model Compression: Apply quantization (INT8), pruning, and knowledge distillation to reduce model size and inference time for mobile/edge deployment while preserving accuracy.

Progressive Inference: Implement cascaded detection where quick spatial checks filter obviously real/fake videos, with expensive temporal analysis reserved for ambiguous cases.

Real-Time Processing: Optimize for video streaming scenarios where frames arrive sequentially, enabling online detection rather than batch processing entire videos.

H. Interpretability and Explainability

Attention Visualization: Implement and visualize attention mechanisms showing which frames and spatial regions most influenced predictions, aiding forensic investigators.

Counterfactual Explanations: Generate minimal perturbations that would flip predictions, revealing which features are critical for detection decisions.

Feature Attribution: Apply techniques like Grad-CAM or SHAP to quantify spatial and temporal feature importance, providing interpretable explanations for forensic documentation.

VII. CONCLUSION

This paper presented a hybrid deepfake detection framework integrating EfficientNet-B3 for spatial feature extraction, Temporal Convolutional Networks for motion pattern recognition, and Exponential Moving Average optimization for training stabilization. The system was specifically designed for resourceconstrained environments, demonstrating that competitive detection accuracy can be achieved without GPU acceleration through architectural efficiency and preprocessing optimization.

A. Key Achievements

Our work makes several contributions to the field of digital media forensics:

1) CPU-Feasible Hybrid Architecture: Demonstrated that combining efficient spatial encoders (EfficientNet-B3) with lightweight temporal models (TCN) enables deepfake detection on standard CPU hardware, broadening accessibility for organizations with limited computational resources.
2) Preprocessing Efficiency: Introduced tensor caching methodology reducing training time by approximately 70% through elimination of redundant face detection across epochs—a simple yet effective optimization often overlooked in research implementations.
3) Systematic Ablation Analysis: Provided comprehensive evaluation isolating contributions of spatial encoding improvements (B0→B3), loss function optimization (BCE→Focal), temporal modeling (TCN integration), and parameter averaging (EMA), validating that each component provides measurable benefits.
4) Transparent Performance Reporting: Unlike many studies reporting only overall accuracy, we provided detailed confusion matrices and per-class metrics, revealing critical performance asymmetries (81.63% real-recall vs. 30.39% fake-recall) that have important implications for practical deployment.
5) EMA Optimization Investigation: Systematically analyzed EMA's impact on deepfake detection, finding that while it provides training stability and reduced variance, aggressive smoothing (=0.9998) may sacrifice accuracy. This finding highlights the need for careful EMA parameter tuning in deployment contexts.

B. Critical Limitations

Honest assessment of limitations is essential for guiding future research and setting realistic expectations:

1) Insufficient Fake-Sample Recall: The most significant limitation is the model's failure to detect 69.6% of manipulated videos (recall: 30.39%). This performance level is inadequate for high-stakes

applications such as legal evidence authentication, election integrity monitoring, or content moderation at scale.

2) EMA-Standard Accuracy Gap: The 11.25 percentage point difference between standard model peak accuracy (96.75%) and EMA final accuracy (85.50%) requires further investigation. While EMA provides stability, the magnitude of this gap suggests potential over-smoothing or insufficient training duration.
3) Limited Evaluation Scope: Testing exclusively on FaceForensics++ C23 limits generalizability claims. Performance on other compression levels, datasets, and real-world video sources remains unknown.
4) Temporal Modeling Underutilization: Despite incorporating TCN, the model may rely predominantly on spatial features (EfficientNet: 12M params vs. TCN: 1.2M params), limiting the realized benefit of temporal analysis.

C. Broader Implications

Beyond technical contributions, this work highlights important considerations for the deepfake detection research community:

Accessibility Matters: By demonstrating CPU-feasible detection, we show that advanced capabilities need not require expensive infrastructure. Democratizing access to detection technology is essential for empowering smaller organizations, developing regions, and independent researchers to combat synthetic media threats.

Metrics Beyond Accuracy: Overall accuracy can mask critical performance asymmetries. The deepfake detection community should standardize reporting of confusion matrices, per-class metrics, and failure mode analysis to enable more realistic assessment of deployment readiness.

Optimization Tradeoffs: Techniques like EMA that improve stability may sacrifice peak performance. Understanding these tradeoffs and choosing appropriate configurations for specific deployment contexts (research benchmarking vs. production systems) is crucial.

The Arms Race Continues: Our low fake-recall, despite sophisticated architecture and optimization, underscores that detection remains fundamentally challenging as synthesis techniques evolve. Sustainable solutions require not just technical detection but also policy frameworks, media literacy, authentication standards, and multi-stakeholder collaboration.

D. Final Remarks

Deepfake technology represents both a remarkable AI achievement and a significant societal challenge. As synthesis methods continue advancing, detection systems must evolve in parallel—becoming more accurate, more efficient, more accessible, and more robust. This work contributes to that evolution by demonstrating

that hybrid architectures combining spatial-temporal analysis with optimized training can deliver competitive performance without prohibitive computational requirements.

However, the critical limitations identified—particularly the low fake-sample recall—underscore that substantial research remains before deepfake detection systems are ready for high-stakes deployment. We hope this work, with its transparent reporting of both successes and failures, provides a realistic foundation for future research while highlighting the most pressing challenges that must be addressed.

The fight against synthetic media manipulation is not solely a technical problem but a societal one requiring collaboration among AI researchers, policymakers, platforms, journalists, and educators. Technical detection tools like ours form one essential component of a comprehensive solution, but they must be complemented by authentication infrastructure, regulatory frameworks, and public awareness to effectively preserve truth and trust in our increasingly AI-mediated digital world.

ACKNOWLEDGMENTS

The authors thank the creators of the FaceForensics++ dataset for providing high-quality benchmark data, the PyTorch and MTCNN library developers for excellent open-source tools, and anonymous reviewers for their constructive feedback that significantly improved this manuscript.

APPENDIX

Preprocessing Code

```
# -*- coding: utf-8 -*-
"""
Preprocessing Script for Deepfake Detector
Extracts faces and caches them as torch tensors (.pt)
Speeds up training by 5–10x on subsequent runs
"""

import os
import cv2
import torch
import numpy as np
from tqdm import tqdm
from pathlib import Path
from facenet_pytorch import MTCNN
from torchvision import transforms

# ==========================
# CONFIG
# ==========================
RAW_REAL_DIR = r"C:\Users\ASUS\OneDrive\Capstone Project\Facenet
Code\original_sequences-20251018T143304Z-1-
001\original_sequences\youtube\c23\videos"
```

```
RAW_FAKE_DIR = r"C:\Users\ASUS\OneDrive\Capstone Project\Facenet
Code\manipulated_sequences-20251018T143143Z-1-
001\manipulated_sequences\Deepfakes\c23\videos"

PROCESSED_ROOT = r"C:\Users\ASUS\OneDrive\Capstone Project\Facenet
Code\processed_faces"
N_FRAMES = 16
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'

# ==========================
# Setup
# ==========================
os.makedirs(os.path.join(PROCESSED_ROOT, "real"), exist_ok=True)
os.makedirs(os.path.join(PROCESSED_ROOT, "fake"), exist_ok=True)

transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

mtcnn = MTCNN(keep_all=False, device=DEVICE)

def extract_and_save(video_path, save_path, n_frames=N_FRAMES):
    """Extracts up to N_FRAMES faces and saves as .pt tensor."""
    cap = cv2.VideoCapture(video_path)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    if total_frames <= 0:
        cap.release()
        print(f"⚠ Skipping empty video: {video_path}")
        return False

    idxs = np.linspace(0, total_frames - 1, n_frames, dtype=int)
    frames, count = [], 0
    frame_id = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break
        if frame_id in idxs:
            frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            face = mtcnn(frame_rgb)
            if face is not None:
                face_np = (face.permute(1, 2, 0).cpu().numpy() * 255).astype(np.uint8)
                frames.append(transform(face_np))
            count += 1
        frame_id += 1
    cap.release()
```

```
    # pad if fewer than n_frames
    while len(frames) < n_frames:
        frames.append(frames[-1] if frames else torch.zeros(3, 224, 224))

    torch.save(torch.stack(frames[:n_frames]), save_path)
    return True

def process_folder(src_dir, dst_dir):
    videos = list(Path(src_dir).glob("*.mp4"))
    print(f" Found {len(videos)} videos in {src_dir}")
    for vid in tqdm(videos, desc=f"Processing {os.path.basename(src_dir)}"):
        out_path = os.path.join(dst_dir, Path(vid).stem + ".pt")
        if os.path.exists(out_path):
            continue  # skip if already processed
        try:
            extract_and_save(str(vid), out_path)
        except Exception as e:
            print(f" Error processing {vid}: {e}")

def main():
    print(f" Starting preprocessing on {DEVICE}")
    process_folder(RAW_REAL_DIR, os.path.join(PROCESSED_ROOT, "real"))
    process_folder(RAW_FAKE_DIR, os.path.join(PROCESSED_ROOT, "fake"))
    print(" Preprocessing complete! Faces cached successfully.")

if __name__ == "__main__":
    main()
```

Training and Validation Code

```
"""
Optimized Deepfake Detector with EMA + Warmup + Evaluation Reports
- Stable version for Windows (no multiprocessing freeze)
- Added: post-training metrics and visualization (confusion matrix, classification
report, training curves)
- Results saved under: training_reports/
"""

import os
import cv2
import torch
import numpy as np
from tqdm import tqdm
from pathlib import Path
from torchvision import transforms
from facenet_pytorch import MTCNN
from efficientnet_pytorch import EfficientNet
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
import torch.nn.functional as F
```

```
import random
import copy
import subprocess
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report

# ===================== CONFIG ====================
REAL_DIR = r"C:\Users\ASUS\OneDrive\Capstone Project\Facenet
Code\original_sequences-20251018T143304Z-1-
001\original_sequences\youtube\c23\videos"
FAKE_DIR = r"C:\Users\ASUS\OneDrive\Capstone Project\Facenet
Code\manipulated_sequences-20251018T143143Z-1-
001\manipulated_sequences\Deepfakes\c23\videos"
CACHE_DIR = r"C:\Users\ASUS\OneDrive\Capstone Project\Facenet
Code\processed_faces"
REPORT_DIR = "training_reports"

N_FRAMES = 16
BATCH_SIZE = 8
EPOCHS = 12
LR = 1e-4
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'

CHECKPOINT_PATH = "checkpoint_latest.pth"
BEST_MODEL_PATH = "best_detector_ema.pth"

FORCE_RETRAIN = False
USE_EMA_AFTER = 2
USE_CUTMIX_AFTER = 2
USE_FOCAL_LOSS = True
BACKBONE = 'efficientnet-b3'

os.makedirs(os.path.join(CACHE_DIR, "real"), exist_ok=True)
os.makedirs(os.path.join(CACHE_DIR, "fake"), exist_ok=True)
os.makedirs(REPORT_DIR, exist_ok=True)

# ===================== Preprocessing ====================
transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                         [0.229, 0.224, 0.225])
])

mtcnn = MTCNN(keep_all=False, device='cpu')  # safer for Windows

def extract_and_cache(video_path, save_path, n_frames=N_FRAMES):
    try:
        if os.path.exists(save_path):
```

```
            return torch.load(save_path)
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            return torch.zeros(n_frames, 3, 224, 224)
        total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
        if total <= 0:
            cap.release()
            return torch.zeros(n_frames, 3, 224, 224)
        idxs = np.linspace(0, total - 1, n_frames, dtype=int)
        frames, fid = [], 0
        while True:
            ret, f = cap.read()
            if not ret:
                break
            if fid in idxs:
                try:
                    f_rgb = cv2.cvtColor(f, cv2.COLOR_BGR2RGB)
                    face = mtcnn(f_rgb)
                    if face is not None:
                        f = (face.permute(1, 2, 0).cpu().numpy() * 255).astype(np.uint8)
                    frames.append(transform(f))
                except Exception:
                    pass
            fid += 1
        cap.release()
        if len(frames) == 0:
            return torch.zeros(n_frames, 3, 224, 224)
        while len(frames) < n_frames:
            frames.append(frames[-1])
        tensor = torch.stack(frames[:n_frames])
        torch.save(tensor, save_path)
        return tensor
    except Exception:
        return torch.zeros(n_frames, 3, 224, 224)

# ==================== Dataset ====================
class CachedDeepfakeDataset(Dataset):
    def __init__(self, real_dir, fake_dir, cache_dir):
        self.real_videos = list(Path(real_dir).glob("*.mp4"))
        self.fake_videos = list(Path(fake_dir).glob("*.mp4"))
        self.items = [(str(v), 0) for v in self.real_videos] + [(str(v), 1) for v in
self.fake_videos]
        self.cache_dir = cache_dir
        print(f"📦 Found {len(self.real_videos)} real, {len(self.fake_videos)} fake
videos")

    def __len__(self): return len(self.items)

    def __getitem__(self, i):
        path, label = self.items[i]
        subdir = "real" if label == 0 else "fake"
```

```
        cache_path = os.path.join(self.cache_dir, subdir, Path(path).stem + ".pt")
        frames = extract_and_cache(path, cache_path)
        return frames, torch.tensor(label, dtype=torch.float32)

# ==================== Model, EMA, Losses ====================
class DeepfakeDetector(nn.Module):
    def __init__(self, backbone_name=BACKBONE, pretrained=True,
tconv_channels=512, dropout=0.1):
        super().__init__()
        self.backbone = EfficientNet.from_pretrained(backbone_name)
        self.feat_dim = self.backbone._fc.in_features
        self.temporal = nn.Sequential(
            nn.Conv1d(self.feat_dim, tconv_channels, kernel_size=5, padding=2,
bias=False),
            nn.BatchNorm1d(tconv_channels),
            nn.ReLU(inplace=True),
            nn.Conv1d(tconv_channels, tconv_channels, kernel_size=5, padding=2,
bias=False),
            nn.BatchNorm1d(tconv_channels),
            nn.ReLU(inplace=True),
        )
        self.pool = nn.AdaptiveAvgPool1d(1)
        self.classifier = nn.Sequential(nn.Dropout(dropout), nn.Linear(tconv_channels,
1))

    def forward(self, x):
        B, T, C, H, W = x.shape
        x = x.view(B * T, C, H, W)
        feat_map = self.backbone.extract_features(x)
        feat = F.adaptive_avg_pool2d(feat_map, 1).view(B, T, self.feat_dim)
        feat = feat.permute(0, 2, 1)
        t = self.temporal(feat)
        t = self.pool(t).view(B, -1)
        return self.classifier(t)

class ModelEMA:
    def __init__(self, model, decay=0.9998):
        self.ema = copy.deepcopy(model).eval()
        for p in self.ema.parameters():
            p.requires_grad_(False)
        self.decay = decay
    def update(self, model):
        with torch.no_grad():
            msd = model.state_dict()
            for k, v in self.ema.state_dict().items():
                if k in msd:
                    v.copy_(v * self.decay + msd[k] * (1.0 - self.decay))

class FocalLoss(nn.Module):
    def __init__(self, alpha=1, gamma=2):
        super().__init__()
```

```
        self.alpha = alpha
        self.gamma = gamma
    def forward(self, inputs, targets):
        bce = F.binary_cross_entropy_with_logits(inputs, targets, reduction='none')
        pt = torch.exp(-bce)
        return (self.alpha * (1 - pt) ** self.gamma * bce).mean()

# ==================== Training / Validation ====================
def train_epoch(model, loader, opt, crit, cutmix, scaler, ema, device,
use_cutmix=False, use_ema=False):
    model.train()
    total_loss, correct = 0, 0
    for x, y in tqdm(loader, desc='Train', leave=False):
        x, y = x.to(device), y.to(device)
        opt.zero_grad()
        with torch.cuda.amp.autocast():
            out = model(x).squeeze(1)
            loss = crit(out, y)
        scaler.scale(loss).backward()
        scaler.unscale_(opt)
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(opt)
        scaler.update()
        if use_ema:
            ema.update(model)
        total_loss += loss.item()
        preds = (torch.sigmoid(out) > 0.5).float()
        correct += (preds == y).sum().item()
    return total_loss / len(loader), 100 * correct / len(loader.dataset)

def validate(model, loader, crit, device, collect_preds=False):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    all_preds, all_labels = [], []
    with torch.no_grad():
        for x, y in tqdm(loader, desc='Val', leave=False):
            x, y = x.to(device), y.to(device)
            out = model(x).squeeze(1)
            loss = crit(out, y)
            total_loss += loss.item() * x.size(0)
            preds = (torch.sigmoid(out) > 0.5).float()
            correct += (preds == y).sum().item()
            total += x.size(0)
            if collect_preds:
                all_preds.extend(preds.cpu().numpy())
                all_labels.extend(y.cpu().numpy())
    avg_loss = total_loss / (total + 1e-12)
    acc = 100.0 * correct / (total + 1e-12)
    if collect_preds:
        return avg_loss, acc, np.array(all_preds), np.array(all_labels)
    return avg_loss, acc
```

```
# ==================== Main ====================
def main():
    # Toggle: also save the best base (non-EMA) model when validation improves
    SAVE_BEST_BASE = True
    BEST_BASE_PATH = "best_detector_base.pth"

    print(f" Using device: {DEVICE}")
    ds = CachedDeepfakeDataset(REAL_DIR, FAKE_DIR, CACHE_DIR)
    n = len(ds)
    if n == 0:
        print("No videos found in given directories. Exiting.")
        return

    # --- Split dataset (80/20) ---
    train_size = int(0.8 * n)
    val_size = n - train_size
    train_ds, val_ds = torch.utils.data.random_split(ds, [train_size, val_size])

    # --- Build a WeightedRandomSampler for balanced training batches ---
    all_labels = [int(lbl.item()) for _, lbl in ds]
    train_indices = train_ds.indices if hasattr(train_ds, "indices") else
list(range(train_size))
    train_labels = [all_labels[i] for i in train_indices]
    class_sample_count = np.array([train_labels.count(t) for t in [0, 1]])
    if class_sample_count.sum() == 0:
        print("Warning: no samples found in training subset.")
        train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,
num_workers=0)
    else:
        weights_per_class = 1.0 / (class_sample_count + 1e-12)
        samples_weights = np.array([weights_per_class[t] for t in train_labels])
        sampler = torch.utils.data.WeightedRandomSampler(samples_weights,
num_samples=len(samples_weights), replacement=True)
        train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, sampler=sampler,
num_workers=0)

    val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False,
num_workers=0)
    print(f" Train batches: {len(train_loader)}, Val batches: {len(val_loader)}")

    # --- Model, EMA, Loss, Optimizer, Scheduler, Scaler ---
    model = DeepfakeDetector().to(DEVICE)
    ema = ModelEMA(model)
    crit = FocalLoss() if USE_FOCAL_LOSS else nn.BCEWithLogitsLoss()
    opt = torch.optim.AdamW(model.parameters(), lr=LR, weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, mode='max',
patience=2, factor=0.5)
    scaler = torch.cuda.amp.GradScaler()

    # --- training tracking ---
```

```
    train_losses, val_losses, train_accs, val_accs = [], [], [], []
    best_acc = 0.0

    # --- Training loop ---
    for e in range(1, EPOCHS + 1):
        print(f"\n Epoch {e}/{EPOCHS}")

        # Train (enable ema updates)
        tr_loss, tr_acc = train_epoch(model, train_loader, opt, crit, None, scaler, ema,
DEVICE, use_ema=True)

        # Validate (base model)
        val_loss, val_acc = validate(model, val_loader, crit, DEVICE)
        scheduler.step(val_acc)

        train_losses.append(tr_loss)
        val_losses.append(val_loss)
        train_accs.append(tr_acc)
        val_accs.append(val_acc)

        print(f"Train Loss: {tr_loss:.4f} | Val Loss: {val_loss:.4f} | Train Acc: {tr_acc:.2f}%
| Val Acc: {val_acc:.2f}%")

        # ------------------ Save checkpoints ------------------
        ckpt = {
            'epoch': e,
            'model_state_dict': model.state_dict(),
            'ema_state_dict': ema.ema.state_dict() if hasattr(ema, "ema") else None,
            'optimizer_state_dict': opt.state_dict(),
            'val_acc': val_acc,
            'train_loss': tr_loss,
            'val_loss': val_loss
        }
        torch.save(ckpt, CHECKPOINT_PATH)
        print(f" Latest checkpoint saved to {CHECKPOINT_PATH}")

        # Save best models (EMA and optional base) when validation accuracy improves
        if val_acc > best_acc:
            best_acc = val_acc
            # Save EMA model weights (recommended)
            if hasattr(ema, "ema"):
                torch.save(ema.ema.state_dict(), BEST_MODEL_PATH)
                print(f" New best EMA model saved (Val Acc: {best_acc:.2f}%) →
{BEST_MODEL_PATH}")
            # Optionally save base (non-EMA) model weights
            if SAVE_BEST_BASE:
                torch.save(model.state_dict(), BEST_BASE_PATH)
                print(f" New best BASE model saved (Val Acc: {best_acc:.2f}%) →
{BEST_BASE_PATH}")
```

```
    # ==================== Final Evaluation (load best model if available)
====================
    print("\n Generating final evaluation report using best model (EMA preferred)...")

    # Prefer EMA best for evaluation; fall back to saved base best if EMA not available;
else in-memory ema; else base in-memory
    eval_model = None
    if os.path.exists(BEST_MODEL_PATH):
        eval_model = DeepfakeDetector().to(DEVICE)
        eval_model.load_state_dict(torch.load(BEST_MODEL_PATH,
map_location=DEVICE))
        eval_model.eval()
        print(f" Loaded best EMA model from {BEST_MODEL_PATH}")
    elif SAVE_BEST_BASE and os.path.exists(BEST_BASE_PATH):
        eval_model = DeepfakeDetector().to(DEVICE)
        eval_model.load_state_dict(torch.load(BEST_BASE_PATH,
map_location=DEVICE))
        eval_model.eval()
        print(f" EMA not found — loaded best BASE model from {BEST_BASE_PATH}")
    else:
        try:
            eval_model = ema.ema
            print("Using in-memory EMA model for final evaluation.")
        except Exception:
            eval_model = model
            print("EMA not available, using base model for final evaluation.")

    # Collect preds & labels for final report
    val_loss, val_acc, preds, labels = validate(eval_model, val_loader, crit, DEVICE,
collect_preds=True)

    # Convert predictions and labels to ints (0/1)
    preds = preds.astype(int)
    labels = labels.astype(int)

    # Print validation summary in desired format
    print(f"Validation Loss: {val_loss:.6f}")
    print(f"Validation Accuracy (%): {val_acc:.4f}\n")

    # Print counts to quickly see predicted distribution
    pred_counts = np.bincount(preds, minlength=2)
    true_counts = np.bincount(labels, minlength=2)
    print(f"Predicted counts - Real: {pred_counts[0]}, Fake: {pred_counts[1]}")
    print(f"True counts      - Real: {true_counts[0]}, Fake: {true_counts[1]}\n")

    # Classification report (zero_division=0 to avoid warnings when a class has no
predictions)
    report = classification_report(labels, preds, target_names=["Real", "Fake"],
digits=4, zero_division=0)
    print(report)
```

```
    # Save classification report
    report_path = os.path.join(REPORT_DIR, "classification_report.txt")
    with open(report_path, "w") as f:
        f.write(f"Validation Loss: {val_loss:.6f}\n")
        f.write(f"Validation Accuracy (%): {val_acc:.4f}\n\n")
        f.write(report)
    print(f" Classification report saved to {report_path}")

    # Confusion matrix and plot
    cm = confusion_matrix(labels, preds)
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=["Real", "Fake"], yticklabels=["Real", "Fake"])
    plt.title("Confusion Matrix (Best Model)")
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.tight_layout()
    cm_path = os.path.join(REPORT_DIR, "confusion_matrix.png")
    plt.savefig(cm_path)
    plt.close()
    print(f" Confusion matrix saved to {cm_path}")

    # Training curves plot
    print("\n Saving training curves...")
    epochs = np.arange(1, len(train_losses) + 1)
    plt.figure(figsize=(10, 4))

    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_losses, label="Training Loss")
    plt.plot(epochs, val_losses, label="Validation Loss")
    plt.title("Loss Curves")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(epochs, train_accs, label="Training Acc")
    plt.plot(epochs, val_accs, label="Validation Acc")
    plt.title("Accuracy Curves")
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy (%)")
    plt.legend()

    plt.tight_layout()
    curves_path = os.path.join(REPORT_DIR, "training_curves.png")
    plt.savefig(curves_path)
    plt.close()
    print(f" Training curves saved to {curves_path}")

if __name__ == "_main_":
    main()
```

REFERENCES

[1] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in Proc. Int. Conf. Machine Learning (ICML), 2019, pp. 6105–6114.

[2] X. Wang, W. Song, C. Hao, and F. Liu, “Deepfake detection method based on spatio-temporal information fusion,” CMC-Computers, Materials & Continua, vol. 83, no. 3, pp. 60590–60612, 2024.

[3] M. Ahmad Amin, Y. Hu, and J. Hu, “Analyzing temporal coherence for deepfake video detection,” Electronic Research Archive, vol. 32, no. 4, pp. 2621–2641, 2024.

[4] R. Oorloff, S. Koppisetti, N. Bonettini, et al., “AVFF: Audio-visual feature fusion for video deepfake detection,” arXiv preprint arXiv:2406.02951, 2024.

[5] C. Hsu, S. Chen, M. Wu, et al., “GRACE: Graph-regularized attentive convolutional entanglement with Laplacian smoothing for robust deepfake video detection,” arXiv preprint arXiv:2406.19941, 2024.

[6] R. Wang, D. Ye, L. Tang, Y. Zhang, and J. Deng, “AVT2-DWF: Improving deepfake detection with audio-visual fusion and dynamic weighting strategies,” arXiv preprint arXiv:2403.14974, 2024.

[7] K. Lipianina-Honcharenko, M. Telka, and N. Melnyk, “Comparison of ResNet, EfficientNet and Xception architectures for deepfake detection,” in Proc.

AdvAIT-2024: 1st Int. Workshop on Advanced Applied Information Technologies, Dec. 2024, pp. 1–12.